



Lesson 11

27.02.2025

```
public class Task1 {  
    public static void main(String[] args) {  
        long thatNumber = 5 ≥ 5 ? 1+2 : 1*1;  
        if(++thatNumber < 4)  
            thatNumber += 1;  
        System.out.println(thatNumber);  
    }  
}
```

```
public class Task2 {  
    public static void main(String[] args) {  
        int meal = 5;  
        int tip = 2;  
        int total = meal + (meal > 6 ? ++tip : --tip);  
        System.out.println(total);  
    }  
}
```

```
public class Task3 {  
    public static String travel(int distance) {  
        return distance<1000 ? "train" : 10;  
    }  
    public static void main(String[] answer) {  
        System.out.print(travel(500));  
    }  
}
```

```
public class Task4 {  
    public static void main(String[] args) {  
        String[] days = new String[]{"Sunday", "Monday", "Tuesday",  
            "Wednesday", "Thursday", "Friday", "Saturday"};  
        for (int i = 0; i < days.size; i++)  
            System.out.println(days[i]);  
    }  
}
```

```
public class Task5 {  
    public static void main(String[] args) {  
        String[] os = new String[] { "Mac", "Windows", "Linux"};  
        System.out.println(Arrays.binarySearch(os, "Linux"));  
    }  
}
```

ООР

У цій парадигмі програмування програма розбивається на об'єкти – структури даних, що складаються з полів, що описують стан, та методів – підпрограм, що застосовуються до об'єктів для зміни чи запиту їх стану. Більшість об'єктно орієнтованих парадигм для опису об'єктів використовуються класи, об'єкти вищого порядку, що описують структуру та операції, пов'язані з об'єктами

РЕАЛІЗАЦІЯ ООР



Encapsulation

Data Security

Inheritance

Code Reusability

OOP Program

Class

Object

Polymorphism

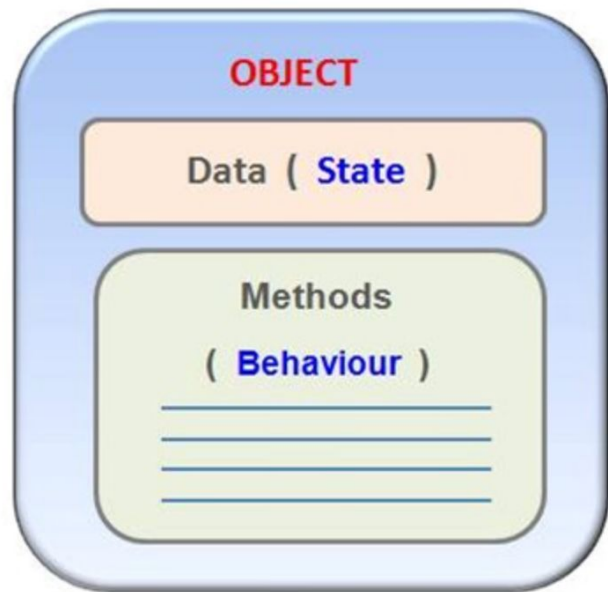
Code Reusability

Abstraction

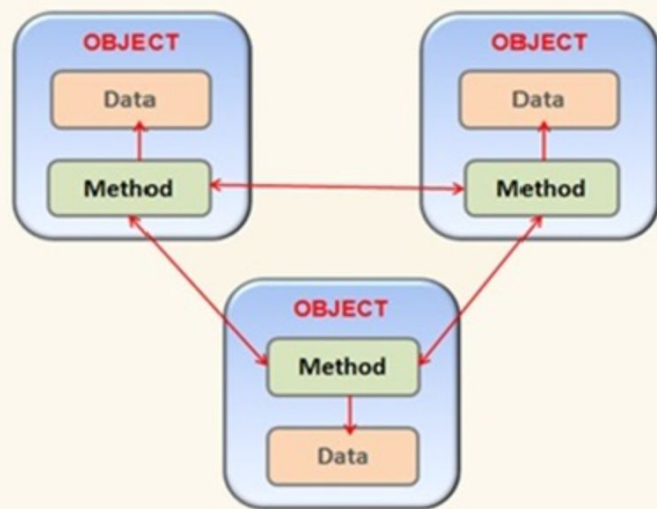
Hides Complexity



OOP - Concept Of Object



OPP - Program Organization





Принципи ООП

Інкапсуляція це властивість системи, що дозволяє об'єднати дані та методи в класі, та приховати деталі реалізації від користувача.

Наслідування це властивість системи, що дозволяє описати новий клас на основі вже існуючого з частково або повністю запозиченої функціональністю

Поліморфізм – один інтерфейс, безліч методів”. Реалізації поліморфізму в мові Java - це навантаження та перевизначення методів, інтерфейси

Абстракція даних це спосіб виділити набір значних показників об'єкта, крім розгляду не значущі. Відповідно, абстракція – це набір всіх таких показників.

ІНКАПСУЛЯЦІЯ

Інкапсуляція — це властивість системи, що дозволяє **об'єднати дані та методи в класі**, та **приховати деталі реалізації** від користувача.



ПЕРЕВАГИ



Захист даних

Запобігає випадковій зміні стану об'єкта.



Контроль доступу

Доступ до даних тільки через визначені методи.

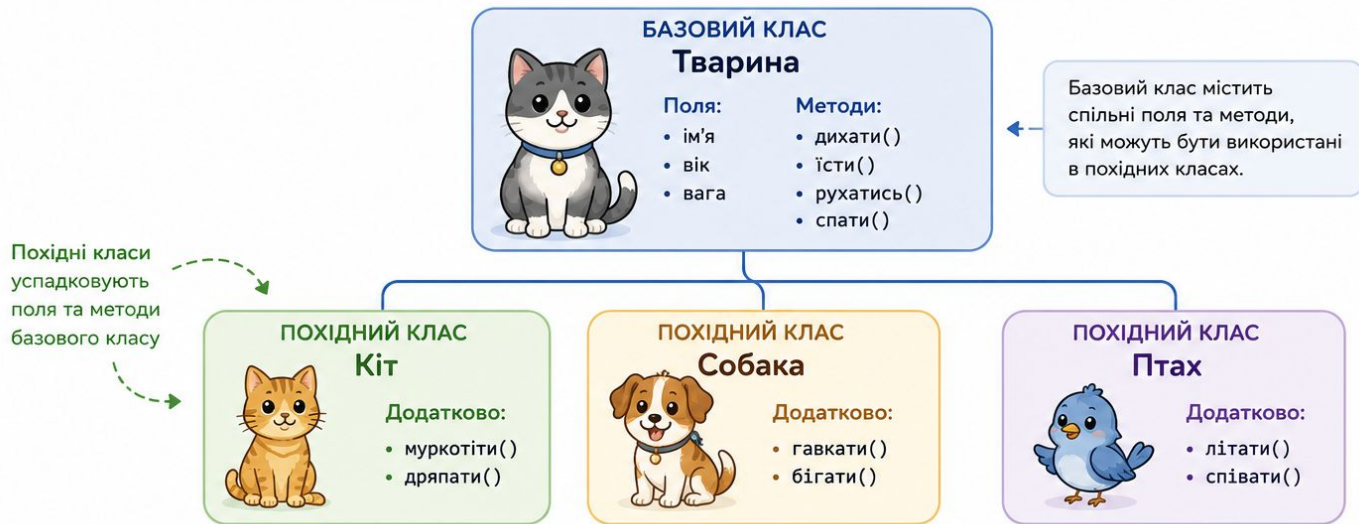


Гнучкість

Можна змінювати реалізацію не впливаючи на користувача.

НАСЛІДУВАННЯ

Наслідування — це властивість системи, що дозволяє описати новий клас на основі вже існуючого з частково або повністю запозиченої функціональності.



Переваги наслідування:

- ✓ Повторне використання коду
- ✓ Легше розширення системи
- ✓ Поліморфізм

Приклад коду (Java):

```
class Тварина {  
    String ім'я;  
    int вік;  
    double вага;  
  
    void дихати() { ... }  
    void їсти() { ... }  
    void рухатись() { ... }  
    void спати() { ... }  
}
```

```
class Кіт extends Тварина {  
  
    void муркотіти() { ... }  
    void дряпати() { ... }  
}
```

```
class Собака extends Тварина {  
  
    void гавкати() { ... }  
    void бігати() { ... }  
}
```

```
class Птах extends Тварина {  
  
    void літати() { ... }  
    void співати() { ... }  
}
```



Похідні класи можуть перевизначати успадковані методи та додавати нову функціональність.

ПОЛІМОРФІЗМ

Поліморфізм – один інтерфейс, безліч методів.

Реалізації поліморфізму в мові Java - це навантаження та перевизначення методів, інтерфейси.

Суть поліморфізму

Один і той самий виклик методу може мати різну реалізацію залежно від типу об'єкта.

```
Animal a;  
a = new Cat();    a.speak(); // May!  
a = new Dog();    a.speak(); // Gав!  
a = new Bird();   a.speak(); // Чік-чіпик!
```

Інтерфейс
(тип)

```
public interface Animal {  
    void speak();  
}
```



Cat

```
@Override  
public void speak() {  
    System.out.println("May!");  
}
```



Dog

```
@Override  
public void speak() {  
    System.out.println("Гав!");  
}
```



Bird

```
@Override  
public void speak() {  
    System.out.println("Чік-чіпик!");  
}
```

РЕАЛІЗАЦІЇ ПОЛІМОРФІЗМУ В JAVA



1. НАВАНТАЖЕННЯ МЕТОДІВ (Method Overloading)

Один клас має декілька методів з однаковим ім'ям, але з різними параметрами (кількість або типи).

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Приклад виклику:

```
Calculator calc = new Calculator();  
calc.add(2, 3);    // int  
calc.add(2.5, 3.5); // double  
calc.add(1, 2, 3); // int (3n.)
```



Вибір потрібного методу відбувається на етапі компіляції на основі кількості та типів параметрів.



2. ПЕРЕВИЗНАЧЕННЯ МЕТОДІВ (Method Overriding)

Підклас надає свою реалізацію методу, успадкованого від батьківського класу.

```
class Animal {  
    void speak() {  
        System.out.println("Тварина видає звук");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void speak() {  
        System.out.println("Гав!");  
    }  
}
```

Приклад:

```
Animal a = new Dog();  
a.speak();  
// Гав!
```



Вибір реалізації методу відбувається на етапі виконання програми (динамічний поліморфізм).



3. ІНТЕРФЕЙСИ (Interfaces)

Різні класи реалізують один інтерфейс, надаючи свою реалізацію методів.

```
interface Animal {  
    void speak();  
}  
  
class Cat implements Animal {  
    public void speak() {  
        System.out.println("May!");  
    }  
}  
  
class Bird implements Animal {  
    public void speak() {  
        System.out.println("Чік-чіпик!");  
    }  
}
```

Приклад:

```
Animal a;  
a = new Cat();  
a.speak(); // May!  
  
a = new Bird();  
a.speak(); // Чік-чіпик!
```



Інтерфейс визначає контракт (що робити), а класи надають власну реалізацію (як робити).



Переваги поліморфізму:



Гнучкість коду



Розширюваність



Повторне використання



Легша підтримка та тестування

АБСТРАКЦІЯ ДАНИХ

Абстракція даних – це спосіб виділити набір **значущих показників** об'єкта, ігноруючи **незначущі деталі**.

Відповідно, **абстракція** – це набір всіх таких показників.

РЕАЛЬНИЙ ОБ'ЄКТ

Об'єкт у реальному світі має багато характеристик, але не всі вони важливі для нашої задачі.

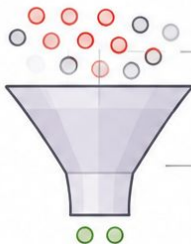


УСІ ХАРАКТЕРИСТИКИ

- Порода
- Колір шерсті
- Кількість шерсті
- Довжина хвоста
- Розмір лап
- Колір очей
- Настрій
- Вік
- Вага
- Улюблена їжа
- ... та інші дрібні деталі

ПРОЦЕС АБСТРАКЦІЇ

Ми виділяємо лише ті показники, які є значущими для вирішення задачі.



Незначущі деталі ігноруються

Значущі показники виділяються

АБСТРАКТНА МОДЕЛЬ

Абстрактна модель містить лише значущі показники об'єкта.



АБСТРАКЦІЯ "КІТ"

- ✓ Ім'я
- ✓ Вік
- ✓ Вага
- ✓ Настрій

Ці показники достатні для нашої задачі.

ПРИКЛАД У JAVA

Реальний об'єкт (з багатьма полями)

```
class Cat {  
    String name;  
    String breed;  
    String furColor;  
    int furLength;  
    int tailLength;  
    String eyeColor;  
    int age;  
    double weight;  
    String mood;  
    String favoriteFood;  
    // ... інші деталі  
}
```

Абстракція –
виділяємо тільки
значущі поля



Абстрактна модель (лише значущі поля)

```
class Cat {  
    private String name;  
    private int age;  
    private double weight;  
    private String mood;  
  
    // Доступ до даних через методи  
  
    public String getName() { ... }  
    public int getAge() { ... }  
    public double getWeight() { ... }  
    public String getMood() { ... }  
}
```

ПЕРЕВАГИ АБСТРАКЦІЇ



Фокус на головному

Ми працюємо лише з тим, що важливо для вирішення задачі.



Спрощення складності

Приховуємо зайві деталі, зменшуємо складність системи.



Легке обслуговування

Зміни в незначущих деталях не впливають на інші частини програми.



Зрозумілий інтерфейс

Користувач взаємодіє з простим та зрозумілим інтерфейсом.



Class vs. Object

ОБ'ЄКТ vs КЛАС

Основні відмінності в ООП



ОБ'ЄКТ



Об'єкт – це екземпляр класу.
Об'єкт створюється на основі класу.



Визначення



Об'єкт – це реальна сутність світу:
наприклад, стілець, ручка, стіл, ноутбук тощо.



Природа



Об'єкт – це фізична сутність.



Тип сутності



Об'єктів можна створювати багато разів
за потреби.



Створення



Об'єкт займає пам'ять при створенні.



Пам'ять



Об'єкт створюється за допомогою ключового слова
new. Наприклад:

```
Cat cat = new Cat();
```



Створення
в коді



Існує декілька способів створити об'єкт у Java:

- за допомогою **new**
- за допомогою методів (**newInstance()**)
- за допомогою **clone()**
- шляхом десеріалізації



Способи
створення



Підсумок:

Клас – це план (опис), а об'єкт – це конкретний екземпляр,
створений за цим планом і який має власний стан та поведінку.



КЛАС



Клас – це «шаблон» (опис),
за яким створюються об'єкти.



Клас – це група схожих об'єктів.
Описує їхні спільні властивості
та поведінку.



Клас – це логічна сутність
(опис у програмі).



Клас оголошується (визначається)
один раз.



Клас не займає пам'ять
при оголошенні.



Клас оголошується за допомогою
ключового слова **class**. Наприклад:



```
class Cat { }
```



Існує лише один спосіб визначення класу –
за допомогою ключового слова **class**.



Приклад на котиках:

Клас **Cat** – це опис усіх котиків.
Об'єкт **cat1**, **cat2**, **cat3** – це конкретні котики,
кожен зі своїм іменем, віком, вагою тощо.





Breed: Bulldog
Size: large
Colour: light gray
Age: 5 years

Dog1Object



Breed: Beagle
Size: large
Colour: orange
Age: 6 years

Dog2Object



Breed: German Shepherd
Size: large
Colour: white & orange
Age: 6 years

Dog3Object

Dog

Fields

Breed
Size
Colour
Age

Methods

Eat()
Run()
Sleep()
Name()

ПАМ'ЯТЬ У JAVA: СТЕК ТА HEAP

У Java об'єкти (посилання) зберігаються в **стеку**, а самі об'єкти – у **heap** (кучі).

1. ОГОВОРЕННЯ ПОСИЛАННЯ ТА ПРИСВОЄННЯ null

```
Box myBox;  
// оголошення посилання
```

```
myBox = null;  
// посилання не вказує  
на жоден об'єкт
```



СТЕК (stack)



HEAP (heap)



Що відбувається?

Створюється посилання `myBox` у стеку. Воно не вказує на жоден об'єкт (`null`). В heap нічого не створюється.



Важливо:

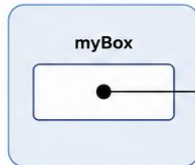
`null` означає відсутність посилання на об'єкт.

2. СТВОРЕННЯ ОБ'ЄКТА ЗА ДОПОМОГОЮ new

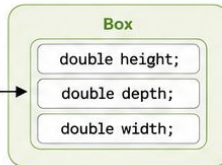
```
myBox = new Box();  
// створення об'єкта Box  
та збереження посилання  
на нього в myBox
```



СТЕК (stack)



HEAP (heap)



Що відбувається?

У heap створюється об'єкт класу `Box` з трьома полями. Посилання `myBox` у стеку тепер вказує на цей об'єкт.



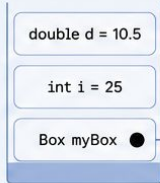
Важливо:

У стеку зберігається тільки посилання, а не сам об'єкт!

СТЕК (stack)

Зберігає:

- примітивні типи (`int`, `double`, `boolean` тощо)
- посилання на об'єкти
- локальні змінні методів

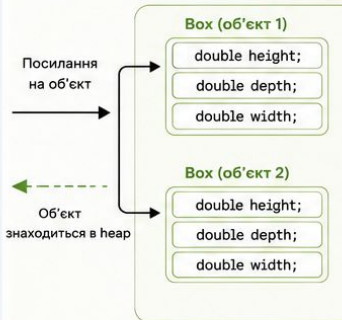


Особливості:



Швидкий доступ.
Пам'ять автоматично звільняється, коли метод завершує роботу.

HEAP (heap)



Зберігає:

- усі створені об'єкти (за допомогою `new`)
- масиви
- рядки (`String`)
- інші складні структури

Особливості:



Більший обсяг, ніж стек.
Пам'ять звільняється Garbage Collector (GC) автоматично.

НА ЩО ЗВЕРНУТИ УВАГУ



Посилання – це змінна, яка містить адресу об'єкта в heap.



Коли посилання = `null` – об'єкт не доступний (якщо немає інших посилань) і буде видалений GC.



Heap живе довше, ніж стек.



Стек для швидких операцій, heap для зберігання даних.





У мові Java під час проектування класів прийнято обмежувати рівень доступу до змінним за допомогою модифікатора **private** і звертатися до них через гетери та сетери.

Існують правила оголошення таких методів, розглянемо їх:

- Якщо властивість НЕ типу `boolean`, префікс геттера має бути `get`. Наприклад: **getName()** - це коректне ім'я геттера для змінної `name`.
- Якщо властивість типу `boolean`, префікс імені геттера може бути `get` або `is`. Наприклад, **getPrinted()** або **isPrinted()** обидва є коректними іменами для змінних типу `boolean`.
- Ім'я сеттера повинне починатися з префікса `set`. Наприклад, **setName()** коректне ім'я для змінної `name`.
- Для створення імені геттера або сеттера, перша літера властивості має бути змінена на велику та додана до відповідного префіксу (`set`, `get` або `is`).
- Сеттер має бути `public`, повертати `void` тип і мати параметр відповідний типу змінної.
- Геттер метод повинен бути `public`, не мати параметрів методу, і повертати значення відповідне типу властивості

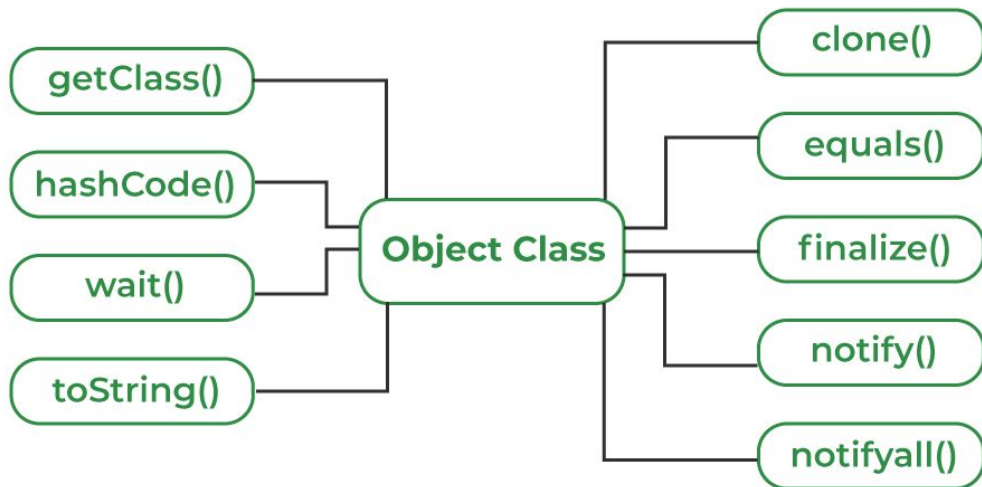
Клас Object

Є суперкласом для всіх класів (включаючи масиви)

Змінна цього типу може посилатися будь-який об'єкт (але не змінну примітивного типу)

Його методи успадковуються усіма класами

Реалізує базові операції з об'єктами





КЛАС Object у JAVA

Клас Object є батьківським для всіх класів у Java.
Він містить базові методи, доступні для кожного об'єкта.



Усі класи в Java
неявно успадковують
методи класу Object.



getClass()

Повертає об'єкт класу Class,
що відповідає класу цього об'єкта.

Повертає: `Class<?>`

```
obj.getClass(); // class java.lang.String
```



hashCode()

Повертає хеш-код (ціле число)
для даного об'єкта.

Повертає: `int`

```
int code = obj.hashCode(); // 12345678
```



wait()

Призупиняє виконання потоку до
отримання повідомлення (notify).

Повертає: `void`

```
obj.wait();
```



toString()

Повертає текстове представлення
об'єкта.

Повертає: `String`

```
String s = obj.toString(); // клас@hashcode
```



Примітка:

Більшість методів класу Object рідко перевизначаються,
але деякі з них (`toString()`, `equals()`, `hashCode()`)
часто перевизначаються у власних класах.

Object Class



clone()

Створює та повертає копію
об'єкта.

Повертає: `Object`

```
Object copy = obj.clone();
```



equals(Object obj)

Порівнює даний об'єкт
з іншим на рівність.

Повертає: `boolean`

```
boolean isEqual = obj.equals(other);
```



finalize()

Викликається перед тим, як об'єкт
буде знищений збирачем сміття.

Повертає: `void`

```
protected void finalize() { ... }
```



notify()

Пробуджує один потік, який
очікує на цьому об'єкті.

Повертає: `void`

```
obj.notify();
```



notifyAll()

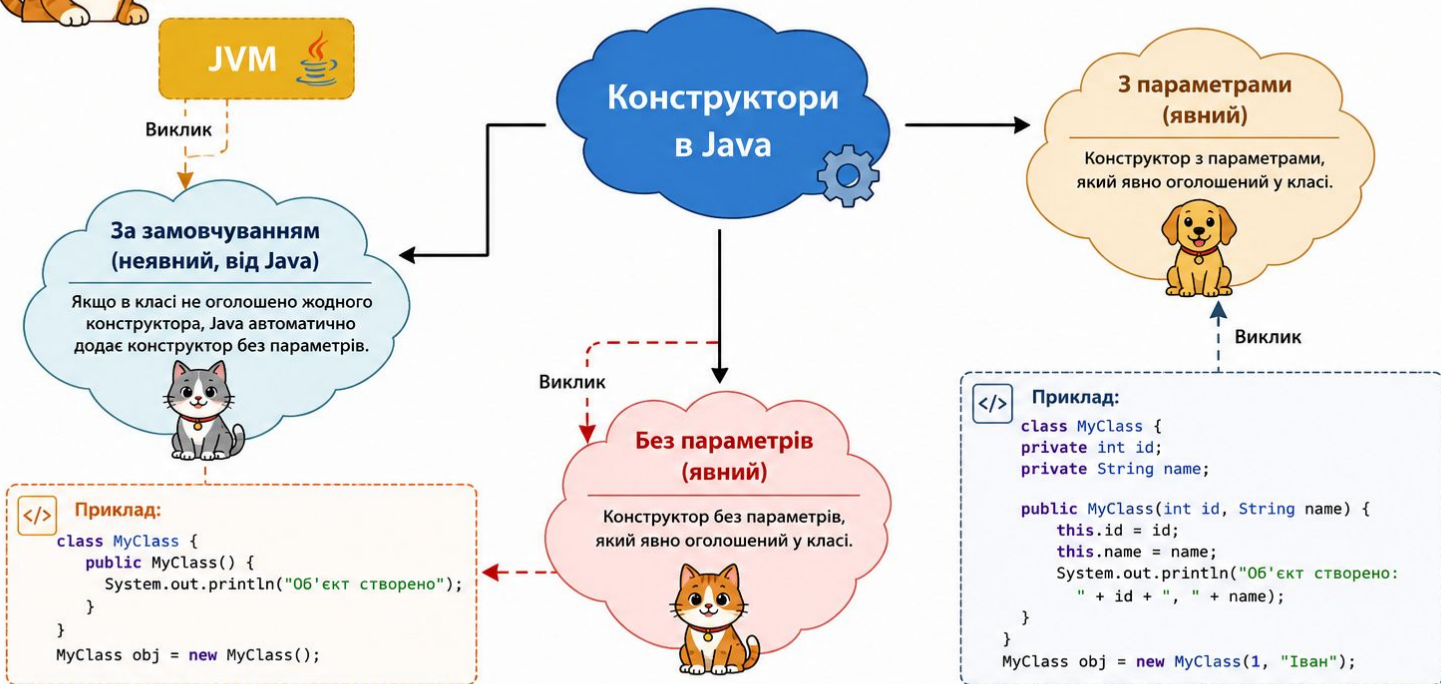
Пробуджує всі потоки, які
очікують на цьому об'єкті.

Повертає: `void`

```
obj.notifyAll();
```


КОНСТРУКТОР У JAVA

Конструктор – це метод, призначення якого полягає у створенні екземпляра класу (об'єкта).



Підсумок:

Конструктори викликаються при створенні об'єкта за допомогою оператора `new`. Вони можуть бути неявними (за замовчуванням) або явними (з параметрами або без них).



Перевантаження функцій (Function Overloading) в програмуванні

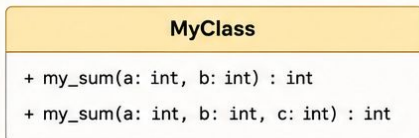


Один і той самий метод може мати кілька версій у межах одного класу з різними параметрами. Компілятор визначає, який метод викликати, залежно від кількості та типів аргументів.

Приклад

```
class MyClass {  
    int my_sum(int a, int b) {  
        return a + b;  
    }  
  
    int my_sum(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Діаграма класу



Ключова ідея:

Методи мають однакове ім'я, але різні списки параметрів (кількість або типи). Повернення типу може бути тим самим або іншим.

Перевизначення функцій (Function Overriding) в програмуванні

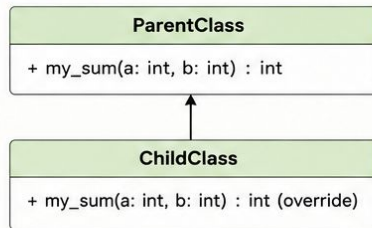


Дочірній клас надає свою реалізацію методу, який уже існує в батьківському класі з таким самим ім'ям, параметрами та типом повернення. Виклик методу залежить від об'єкта.

Приклад

```
class ParentClass {  
    int my_sum(int a, int b) {  
        return a + b;  
    }  
}  
  
class ChildClass extends ParentClass {  
    @Override  
    int my_sum(int a, int b) {  
        return a - b; // перевизначена реалізація  
    }  
}
```

Діаграма класів



Ключова ідея:

Метод у дочірньому класі має ту саму сигнатуру (ім'я, параметри, тип повернення), що й у батьківському класі.